

B.TECH 1ST YEAR · 1ST SEMESTER

C Programming

Formula Sheet & Revision Document

UNIT IV — Pointers & User Defined Data Types

 Pointers & Addresses

 Pointer Arithmetic

 Arrays via Pointers

 Call by Reference

 Structures

 Unions

 Flashcards

 Exam Tips

Last-Day Revision Weapon · Exam-Optimized · 15 Pages

Table of Contents

01	Introduction to Pointers	02	Declaration & Initialization
03	Dereferencing & Address Operators	04	Pointer Arithmetic
05	Pointers and Arrays	06	Array Manipulation via Pointers
07	Pointers with Functions	08	Structures
09	Structure with Pointers	10	Array of Structures
11	Unions	12	Structure vs Union
13	Comparison Tables	14	Key Terms & Definitions
15	Formula / Syntax Box	16	Problem-Solving Patterns
17	Flashcards (25 Q&A)	18	Exam Tips
19	Quick Revision Sheet		

1 Introduction to Pointers

Definition: A **pointer** is a variable that stores the *memory address* of another variable, instead of storing a data value directly.

Why Pointers?

- Direct memory access & manipulation
- Pass large data to functions efficiently
- Dynamic memory allocation
- Build complex data structures (linked lists, trees)
- Return multiple values from functions

Advantages

- Faster execution (no data copying)
- Efficient array & string handling
- Enables dynamic memory (malloc/free)
- Essential for system/embedded programming

Pointer Concept Tree

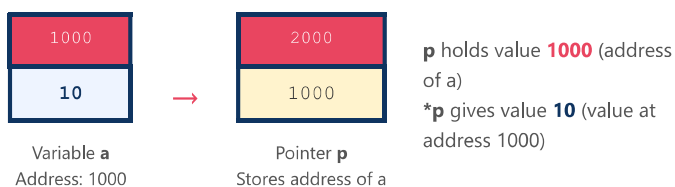
POINTER |— **Stores Address** → Points to a memory location |— **Enables Indirect Access** → Access variable via its address |— **Dynamic Memory** → malloc(), calloc(), free() |— **Pointer Arithmetic** → p++, p--, p+n |— **Function Interaction** → Call by Reference

2 Pointer Declaration & Initialization

Syntax

```
data_type *pointer_name; // Declaration
int a = 10; int *p; // p is a pointer to int
p = &a; // p stores address of a
```

Memory Diagram



Declaration	Meaning	Example
int *p	Pointer to int	p = &x;

float *fp	Pointer to float	fp = &f;
char *cp	Pointer to char	cp = &c;
void *vp	Generic pointer	vp = &any;
int **pp	Pointer to pointer	pp = &p;

3 Dereferencing & Address Operators

& → Address-of operator Returns the memory address of a variable Example: `p = &a;` // p gets address of a

***** → Dereference (Indirection) operator Accesses the value at the stored address Example: `printf("%d", *p);` // prints 10

```
int a = 10; int *p = &a; printf("Address of a : %p\n", &a); // e.g. 0x1000 printf("Value of p : %p\n", p); // e.g. 0x1000 (same) printf("Value of a : %d\n", a); // 10 printf("Value via *p : %d\n", *p); // 10 (dereferenced) *p = 20; // Changes a to 20 via pointer printf("New a = %d\n", a); // 20
```

⚠ Key Rule: Always initialize a pointer before dereferencing. An uninitialized pointer causes **undefined behavior** (crash or garbage output).

4 Pointer Arithmetic

Key Concept: When a pointer is incremented/decremented, it moves by **sizeof(data_type)** bytes — NOT by 1 byte.

$p + n$ → moves to address: $\text{base_address} + (n \times \text{sizeof}(\text{type}))$ $p++$ → $p = p + \text{sizeof}(\text{int}) = p + 4$ (for int on 32-bit) $p--$ → $p = p - \text{sizeof}(\text{int})$ $p1 - p2$ → number of elements between two pointers

Example (int pointer, size = 4 bytes)

```
int arr[] = {10, 20, 30}; int *p = arr; // p → arr[0], address 1000 p++; // p → arr[1], address 1004 p++; // p → arr[2], address 1008 printf("%d", *p);  
// prints 30
```

Operation	Meaning	Result (int*, base=1000)
p++	Increment pointer	1000 → 1004
p--	Decrement pointer	1004 → 1000
p + 2	Move 2 elements forward	1000 → 1008
p - 1	Move 1 element backward	1008 → 1004
p1 - p2	Distance in elements	$(1008 - 1000) / 4 = 2$
*p++	Get value, then advance	value at p, then p moves

✗ **NOT allowed:** Addition of two pointers ($p1 + p2$), Multiplication/Division of pointers, Adding float to pointer.

5 Pointers and Arrays

Golden Rule: Array name = Pointer to first element. $\text{arr} \equiv \&\text{arr}[0]$

```
arr[i] = *(arr + i) // Equivalent notations &arr[i] = (arr + i) // Address of i-th element arr[0] = *arr //  
First element
```

Memory Layout – int arr[4] = {10, 20, 30, 40}

Index	Expression	Address	Value
arr[0]	$\&\text{arr}[0]$ or arr	1000	10
arr[1]	$\&\text{arr}[1]$	1004	20
arr[2]	$\&\text{arr}[2]$	1008	30
arr[3]	$\&\text{arr}[3]$	1012	40

```
int arr[] = {10, 20, 30, 40}; int *p = arr; for(int i=0; i<4; i++) printf("%d ", *(arr+i)); // OR p[i] OR *(p+i)
```

6 Array Manipulation Using Pointers

① **Traversal Using Pointer**

```
int arr[5] = {1,2,3,4,5}; int *p = arr; for(int i=0; i<5; i++) printf("%d ", *p++); // Prints: 1 2 3 4 5
```

② Sum of Array Using Pointer

```
int sum = 0, *p = arr; for(int i=0; i<5; i++) sum += *p++; printf("Sum = %d\n", sum);
```

③ Finding Max Using Pointer

```
int *p = arr, max = *p; for(int i=1; i<5; i++) { if(*(p+i) > max) max = *(p+i); } printf("Max = %d\n", max);
```

7 Pointers with Functions

Feature	Call by Value	Call by Reference
How it works	Copy of value passed	Address of variable passed
Original modified?	✗ No	✓ Yes
Parameter type	Normal variable	Pointer variable
Memory	Extra copy created	No extra copy
Syntax	<code>func (a, b)</code>	<code>func (&a, &b)</code>

Swap Using Pointers (Call by Reference)

```
void swap(int *a, int *b) { int temp = *a; *a = *b; *b = temp; } int main() { int x = 5, y = 10; swap(&x, &y); // Pass addresses printf("%d %d\n", x, y); // 10 5 }
```

Trace: `swap(&x, &y)` → `a=&x, b=&y` → `*a=5, *b=10` → `temp=5, *a=10, *b=5` → `x=10, y=5` ✓

8 Structures

Definition: A **structure** is a user-defined data type that groups variables of *different data types* under a single name.

Syntax & Declaration

```
struct student { // Structure definition int id; // Member 1 char name[20]; // Member 2 float marks; // Member 3 }; // Semicolon mandatory!  
struct student s1; // Variable declaration struct student s1 = {1, "Ravi", 85.5}; // Initialization
```

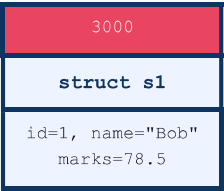
Accessing Structure Members

```
s1.id = 101; // Dot operator (direct access) strcpy(s1.name, "Alice"); s1.marks = 92.5; printf("ID: %d, Name: %s, Marks: %.1f", s1.id, s1.name, s1.marks);
```

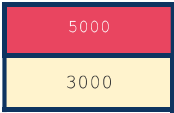
Operator	Used With	Syntax	Example
.	Structure variable	<code>var.member</code>	<code>s1.id</code>
->	Pointer to structure	<code>ptr->member</code>	<code>ptr->id</code>

9 Structure with Pointers

```
struct student s1 = {1, "Bob", 78.5}; struct student *ptr = &s1; // Pointer to structure printf("%d\n", ptr->id); // Arrow operator printf("%s\n", ptr->name); printf("%.1f\n", ptr->marks); // (*ptr).id is same as ptr->id
```



Structure s1



ptr (stores addr 3000)

ptr->id = 1
(*ptr).name = "Bob"

10 Array of Structures

```
struct student { int id; char name[20]; float marks; }; struct student s[3] = { // Array of 3 structures {1, "Alice", 88.5}, {2, "Bob", 79.0}, {3, "Carol", 92.5} }; for(int i=0; i<3; i++) printf("%d %s %.1f\n", s[i].id, s[i].name, s[i].marks);
```

Access Style	Expression	Meaning
Direct (array index)	<code>s[i].id</code>	id of i-th student
Via pointer	<code>(s+i)->id</code>	Same, using arrow
Dereference	<code>(* (s+i)).id</code>	Same, dereferenced

11 Unions

Definition: A **union** is a user-defined data type where all members *share the same memory location*. Only one member can hold a value at a time.

Syntax

```
union data { int i; // 4 bytes float f; // 4 bytes char c; // 1 byte }; //  
Size = max(4,4,1) = 4 bytes union data u; u.i = 10; printf("%d", u.i);  
// OK u.f = 3.14; // Overwrites i!
```

Shared Memory Diagram



All share same 4 bytes

⚠ Only last assigned member gives valid result

12 Structure vs Union — Comparison Table

Feature	Structure	Union
Keyword	<code>struct</code>	<code>union</code>
Memory	Sum of all member sizes	Size of largest member only
Member storage	Each member has own space	All members share same space
Active members	All can be used at once	Only one at a time
Size (int+float+char)	4+4+1 = 9 (+ padding)	max(4,4,1) = 4 bytes
Modification	Changing one does NOT affect others	Changing one OVERWRITES all
Use case	Record with multiple fields	Memory-efficient variant type

13 Important Comparison Tables

① Pointer vs Array

Feature	Pointer	Array
Type	Variable (stores address)	Constant pointer (fixed address)
Memory	Dynamically allocated	Statically allocated
Reassignment	✓ p = &b (reassignable)	✗ arr = &b (illegal)
sizeof	Size of pointer (4 or 8 bytes)	Total array size
Increment	✓ p++ (valid)	✗ arr++ (invalid)

② Call by Value vs Call by Reference

Aspect	Call by Value	Call by Reference
Passing mechanism	Copy of actual value	Address of variable
Original variable	✗ Not changed	✓ Changed
Memory usage	More (duplicate copy)	Less (same memory)
Speed	Slower (large structs)	Faster
Safety	Safer (no side effects)	Must be careful
Example	func(x)	func(&x)

③ Dot (.) vs Arrow (->) Operator

Feature	Dot Operator (.)	Arrow Operator (->)
Used with	Structure variable	Pointer to structure
Syntax	struct_var.member	ptr->member
Equivalent	—	(*ptr).member
Example	s1.id	ptr->id

④ * (Asterisk) vs & (Ampersand)

Operator	Name	Function	Example	Result
*	Dereference / Indirection	Gets VALUE at address	*p	Value stored at p's address
&	Address-of	Gets ADDRESS of variable	&a	Memory address of a
* (decl)	Pointer declaration	Declares pointer variable	int *p	p is pointer to int

⑤ struct vs typedef struct

Feature	struct	typedef struct
Declaration	struct student s1;	Student s1;
Tag required	✓ Yes (struct keyword)	✗ No (alias used)
Readability	Verbose	Cleaner

Example	<code>struct node n;</code>	Node n;
---------	-----------------------------	---------

14 Key Terms & Definitions (30+ One-Liners)

Term	Definition
Pointer	Variable that stores memory address
Dereferencing	Accessing value via pointer using *
Address-of (&)	Returns memory address of a variable
NULL Pointer	Pointer that points to address 0 (nothing)
Wild Pointer	Uninitialized pointer (undefined behavior)
Dangling Pointer	Pointer pointing to freed/deleted memory
Void Pointer	Generic pointer (void *), can hold any address
Double Pointer	Pointer to a pointer (int **pp)
Pointer Arithmetic	Moving pointer by sizeof(type) steps
Array Name	Constant pointer to first array element
Dynamic Memory	Memory allocated at runtime via malloc/calloc
malloc()	Allocates n bytes; returns void pointer
free()	Releases dynamically allocated memory
Call by Value	Function receives copy; original unchanged
Call by Reference	Function receives address; original changed

Term	Definition
Structure	Collection of different data types under one name
Union	Data type where members share same memory
Member	Individual variable inside struct/union
Dot Operator (.)	Accesses struct member via variable
Arrow Operator (->)	Accesses struct member via pointer
typedef	Creates alias/alternate name for a type
Nested Structure	Structure inside another structure
Array of Structures	Array where each element is a struct
Structure Pointer	Pointer that holds address of a structure
sizeof()	Returns size of type/variable in bytes
Padding	Extra bytes added to align struct members
Enumeration (enum)	Named integer constants
Bit Field	Struct member with specified number of bits
Scope	Region of program where variable is accessible
Indirection	Accessing data via pointer (indirect access)

15 Formula / Syntax Box

```
/* — POINTER — */ data_type *ptr; Pointer declaration ptr = &variable; Store address *ptr Dereference (get value) ptr + n → addr + n*sizeof(type) Pointer arithmetic /* — STRUCTURE — */ struct tag { members; }; Define structure struct tag var; Declare variable var.member Access via variable ptr->member = (*ptr).member Access via pointer /* — UNION — */ union tag { members; }; Define union sizeof(union) = max member size Size rule /* — ARRAY & POINTER EQUIV — */ arr[i] ≡ *(arr+i) Element access &arr[i] ≡ arr+i Address access
```

16 Problem-Solving Patterns

① Pointer Traversal Pattern

```
int *p = arr; // Start at beginning while(p < arr + n) { // Loop until end /* process *p */ p++; // Advance pointer }
```

② Swapping Using Pointers

```
void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; // Classic 3-step swap }
```

③ Structure Handling Pattern

```
struct student { int id; char name[20]; float marks; }; void display(struct student *s) { // Pass by pointer printf("%d %s %.1f\n", s->id, s->name, s->marks); }
```

④ Nested Structures

```
struct date { int d, m, y; }; struct student { int id; struct date dob; // Nested structure }; struct student s; s.dob.d = 15; s.dob.m = 8; s.dob.y = 2003;
```

⑤ Dynamic Array Using Pointer (malloc)

```
int n = 5; int *arr = (int*) malloc(n * sizeof(int)); // Allocate for(int i=0; i<n; i++) arr[i] = i*10; // Use like array free(arr); // Always free!
```

⑥ Common Pointer Expressions Decoded

Expression	Meaning
*p++	Get value at p, THEN increment p
(*p)++	Increment VALUE at p (not address)
++*p	Increment value at p (pre-increment)
*++p	Increment p first, THEN get value
**pp	Dereference a pointer to pointer
p[i]	Same as *(p+i)

17 Flashcards — Quick Q&A (25 Cards)

Q: What is a pointer?

A: Variable that stores memory address of another variable

Q: What does * do in expression?

A: Dereferences pointer – accesses value at address

Q: What is a NULL pointer?

A: Pointer initialized to 0/NULL – points to nothing

Q: What is a dangling pointer?

A: Pointer pointing to already freed memory

Q: If `p = 1000 (int*)`, what is `p+1`?

A: 1004 (int is 4 bytes, so `+1 = +4 bytes`)

Q: What is call by reference?

A: Passing address of variable so function can modify original

Q: Dot vs Arrow operator?

A: Dot (.) for variable; Arrow (->) for pointer to struct

Q: What is a union?

A: Data type where all members share the same memory location

Q: Size of structure?

A: Sum of all member sizes (may include padding)

Q: How to declare pointer to struct?

A: `struct student *ptr = &s1;`

Q: Can you add two pointers?

A: No – only subtraction of pointers is valid

Q: What is typedef?

A: Creates an alias for an existing data type

Q: What is nested structure?

A: A structure defined as a member inside another structure

Q: What does * do in declaration?

A: Declares a pointer variable (e.g., `int *p`)

Q: What does & operator do?

A: Returns the memory address of a variable

Q: What is a wild pointer?

A: Pointer declared but not initialized – unsafe

Q: `arr[i]` is equivalent to?

A: `*(arr + i)`

Q: What does `p++` do?

A: Moves pointer to next element (adds `sizeof(type)`)

Q: What is a structure?

A: User-defined type grouping variables of different types

Q: `ptr->member` is same as?

A: `(*ptr).member`

Q: Size of union?

A: Size of its largest member

Q: How many members active in union?

A: Only one at a time (last assigned is valid)

Q: What is `sizeof(int)`?

A: Typically 4 bytes (on 32/64-bit systems)

Q: What does `void*` mean?

A: Generic pointer – can point to any data type

Q: Array name is equivalent to?

A: Constant pointer to first element (`&arr[0]`)

✗ Common Mistakes to Avoid

- Using uninitialized (wild) pointer → crash
- Confusing * (dereference) with & (address-of)
- Writing arr++ instead of using a separate pointer
- Forgetting semicolon after struct/union definition
- Using . instead of -> for structure pointer
- Not freeing dynamically allocated memory
- Reading union member that wasn't last written
- Wrong format specifier %d vs %p for pointers
- Adding two pointers (illegal operation)
- Confusing sizeof(pointer) with sizeof(array)

✓ Exam Writing Tips

- Always draw memory diagrams for pointer questions
- Show address-value relationships in boxes
- Trace pointer arithmetic step-by-step
- Write proper indentation in code answers
- Mention "sizeof" when explaining pointer movement
- Use correct syntax: int *p; NOT int* p (both valid, but int *p is clearer)
- For struct: mention dot vs arrow with examples
- Include output/trace in programs where asked
- Label diagrams clearly (variable name + address)
- Use typedef for cleaner code in answers

Tricky Output Questions — Practice These

```
int a=5, *p=&a; (*p)++; printf("%d", a); // Output: 6
```

```
int arr[]={1,2,3}; int *p=arr; printf("%d", *p++); // Output: 1 (p now at arr[1])
printf("%d", *p); // Output: 2
```

```
union u { int i; float f; }; union u x; x.i = 10; x.f = 3.5; printf("%d", x.i);
// Garbage! f was last set
```

```
struct s { int x; int y; }; struct s a = {3, 7}; struct s *p = &a;
printf("%d", p->x + p->y); // Output: 10
```

⚡ **High-Probability Exam Topics:** Pointer to structure (arrow operator), swapping via pointers, array traversal via pointer, union memory sharing, structure vs union differences, pointer arithmetic with output tracing.

🔴 Rules Only · No Explanations · Pure Bullet Points

📌 Pointer Syntax

- `int *p;` – declare
- `p = &a;` – store address
- `*p` – get value (dereference)
- `p++` → `p += sizeof(int)` bytes
- `arr` ≡ `&arr[0]`
- `arr[i]` ≡ `*(arr+i)`
- NULL pointer = 0 / nothing
- `void *p` → generic pointer
- `int **pp` → pointer to pointer

🏠 Structure Syntax

- `struct tag { members; };`
- `struct tag var = { values };`
- `var.member` → dot (variable)
- `ptr->member` → arrow (pointer)
- `(*ptr).member` ≡ `ptr->member`
- `struct tag arr[n]` → array of structs
- Size = sum of members + padding
- Nested: `s.dob.year`
- `typedef struct {} Alias;`

🔄 Union Syntax

- `union tag { members; };`
- All members share same memory
- Size = `sizeof(largest member)`
- Only one member valid at a time
- Last assigned = valid member

🔢 Pointer Arithmetic

- `p+n` → skips `n` elements
- `p1 - p2` → elements between
- `p++` / `p--` → move by `sizeof`
- `*p++` → value, then advance
- `(*p)++` → increment value

⚖️ Key Differences

struct vs union:

- struct: separate memory per member
- union: shared memory for all

. vs ->:

- `.` : variable access
- `->` : pointer access

*** vs &:**

- `*` : get value at address
- `&` : get address of variable

val vs ref:

- `func(x)` → copy, no change
- `func(&x)` → address, changes `x`

```
/* == ULTRA-FAST REFERENCE == */ int *p = &a; → p stores address of a, *p gives value p++ → address
increases by sizeof(int) = 4 bytes arr[i] ≡ *(arr+i) ≡ *(p+i) ≡ p[i] s1.id → struct variable + dot ptr->id →
struct pointer + arrow (same as (*ptr).id) sizeof(struct) = sum of members | sizeof(union) = max member
```

🎯 All the best for your exam! You've got this! 🚀

C Programming · Unit IV · Pointers & User Defined Data Types